

PROJET HARDENING LINUX - TP M2

Rapport Technique et Bilan de la Semaine 2

1. Introduction et Contexte

Ce rapport technique presente les realisations et developpements effectues pour le Sprint 2 (Semaine 2) dans le cadre de la mise en oeuvre du TP de Hardening Linux pour le Master 2 SRS.

L'objectif de cette semaine etait d'établir l'infrastructure de base contenant les premieres failles de securite et de developper le Robot de controle charge d'auditer et evaluer en temps reel l'etat de la machine de l'etudiant. Deux types de failles majeures ont ete cibles :

- Les vulnerabilites d'accès SSH (Port standard, Root login autorise, Mot de passe faible, et Mots de passe vides).
- L'exposition de ports reseaux inutiles et dangereux (Service rpcbind sur le port 111).

Le developpement a respecte la repartition des taches ('lane David') ainsi que les specifications et coefficients valides dans le programme de la semaine.

2. Conteneur Vulnerable & Dockerfile v1

Le conteneur cible represente l'environnement de l'etudiant. Il a ete renforce de maniere a presenter deliberement les vulnerabilites suivantes :

1. Root password faible : configure avec le mot de passe 'toor'.
2. PermitRootLogin yes : autorise la connexion SSH directe du compte root avec mot de passe.
3. PasswordAuthentication yes : autorise l'authentification SSH globale par mot de passe.
4. PermitEmptyPasswords yes : autorise les connexions pour des comptes sans mot de passe.
5. Service rpcbind actif : installe et demarre en tâche de fond pour ouvrir inutilement le port 111.

Le fichier Dockerfile ('vulnerable/Dockerfile') a ete modifie comme suit :

```
FROM ubuntu:24.04
RUN apt-get update && apt-get install -y openssh-server rpcbind \
    && rm -rf /var/lib/apt/lists/*
RUN mkdir /var/run/ssh
RUN useradd -m -s /bin/bash student && \
    echo "student:student" | chpasswd && \
    echo "root:toor" | chpasswd
COPY authorized_keys /root/.ssh/authorized_keys
RUN chmod 600 /root/.ssh/authorized_keys
RUN sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config && \
    sed -i 's/#PasswordAuthentication yes/PasswordAuthentication yes/' /etc/ssh/sshd_config && \
    echo "PermitEmptyPasswords yes" >> /etc/ssh/sshd_config
EXPOSE 22 111
CMD ["/bin/sh", "-c", "rpcbind && mkdir -p /var/run/ssh && /usr/sbin/sshd -D"]
```

3. Robot de Controle et Scripts de Verification

Le Robot de controle s'execute au sein de l'environnement de management et n'est pas installe sur la machine de l'etudiant. Il se connecte via une cle de secours ED25519 (backdoor admin) creee a cet effet et deposee dans le conteneur cible.

Deux verifications distinctes ont ete ecrites en Node.js/TypeScript :

a) Faille SSH (check_ssh_password.ts) :

Le robot tente de se connecter en root via la cle de secours, puis verifie :

- Si le hash du mot de passe de l'etudiant dans /etc/shadow a ete modifie par rapport a la valeur par default.
- Les parametres actifs de sshd en executant la commande 'sshd -T' (recuperation de PermitRootLogin, PasswordAuthentication, et PermitEmptyPasswords). La faille est consideree comme resolue (PATCHED) uniquement si le mot de passe a change ET que la configuration SSHD est securisee.

b) Faille Ports Ouverts (check_unused_ports.ts) :

Le robot tente d'ouvrir une socket de connexion reseau brute sur le port 111 (rpcbind). Si la connexion reussit, le service est declare expose (VULNERABLE). Si la connexion echoue (timeout ou refused), le service est considere comme ferme (PATCHED).

4. Strategie de Scoring et Base de Donnees

Conformement aux coefficients definis dans le programme de la semaine 2 :

- La faille SSH possede un coefficient de x3 (criticite elevee, faille la plus exploitable).
- La faille de ports ouverts possede un coefficient de x2 (surface reseau elargie).

La base de donnees a ete alimentee dynamiquement via le script de seeding ('database/seeds/seed.js') avec les coefficients correspondants. Le Robot asynchrone recupere ces regles et calcule le score global de l'etudiant sur un maximum de 500 points (100 pts de base * coefficient) normalise a la reduction de surface d'attaque reseau.

5. Verification et Validation Technique

La validation technique a ete realisee de bout en bout en local en simulant le comportement d'un etudiant :

1. Etat Vulnerable Initial :

L'environnement démarre. Une inscription d'étudiant de test est associée à l'IP du conteneur vulnérable.

Les logs du Robot affichent :

- check_ssh_password | VULNERABLE (X)
- check_unused_ports | VULNERABLE (X)
- Score recalcule : 0 / 500 points

2. Resolution de la faille de ports (rpcbind) :

Nous avons simulé la résolution de la faille en tuant le daemon rpcbind dans le conteneur :

Command : docker exec hardening_internship-vulnerable-1 pkill rpcbind

Au cycle suivant, le Robot s'exécute et valide immédiatement le patch :

```
[RobotService] Robot démarre
[RobotService] enrollment=1 | check_ssh_password | X
[RobotService] enrollment=1 | check_unused_ports | X
[RobotService] enrollment=1 | score=0/500
...
[RobotService] enrollment=1 | check_ssh_password | X
[RobotService] enrollment=1 | check_unused_ports | [OK]
[RobotService] enrollment=1 | score=200/500
```

6. Conclusion et perspectives

L'objectif de cette deuxième semaine est pleinement atteint. L'infrastructure Docker-Compose, le Robot asynchrone de contrôle et la base de données de scoring fonctionnent en parfaite synergie. Les premiers checks réseaux et système sont robustes et réactifs. La suite du projet (Semaine 3) visera à ajouter les failles applicatives AppArmor et les permissions SUID abusives.